VitalFlow V1 — Encounter Workspace

Design doc + scaffold audit for the per-encounter workspace at `/encounters/[id]`. Most of the workspace shipped in Slices 3 + 4 and the diagnoses/insurance PRs; this document formalizes the architecture and fills the four remaining sections (Allergies, Medications, Documents, Audit trail).

See also:

- docs/clinical-domain.md — entity shapes
- docs/audit-logging.md — audit taxonomy
- docs/permissions-matrix.md — permission gates
- docs/patient-appointment-ui.md — upstream patient + appointment flows

---

1. Page architecture

Route: `/encounters/[id]` — server component, `force-dynamic` so session-derived permissions and fresh data render every request.

The workspace is a vertical stack of Cards. No tabs — clinicians skim top-to-bottom in safety-first order:

```
┌──────────────────────────────────────────────────────────────┐
│  PageHeader:  <patient>  MRN  DOB age  sex  pronouns  [status] │
│  actions:  [Open chart]                                       │
├──────────────────────────────────────────────────────────────┤
│  Card: Visit summary            (chief complaint, status, reason)│
├──────────────────────────────────────────────────────────────┤
│  Card: Allergies         🆕      (safety-critical — second)       │
├──────────────────────────────────────────────────────────────┤
│  Card: Medications       🆕      (safety-critical — third)        │
├──────────────────────────────────────────────────────────────┤
│  Card: Diagnoses                (ICD-10 mapping for this visit) │
├──────────────────────────────────────────────────────────────┤
│  Card: Vitals                   (timeline + entry)              │
├──────────────────────────────────────────────────────────────┤
│  Card: Clinical note            (SOAP editor + Sign + Amend)    │
├──────────────────────────────────────────────────────────────┤
│  Card: Documents         🆕      (read-only list, upload deferred)│
├──────────────────────────────────────────────────────────────┤
│  Card: Version history          (per-version metadata)          │
├──────────────────────────────────────────────────────────────┤
│  Card: Note audit trail  🆕      (gated on audit:read)           │
└──────────────────────────────────────────────────────────────┘
```

Safety ordering rationale: the prescriber needs allergies and current medications visible *before* they write orders or sign a plan. Putting them near the top reduces the chance of missed contraindications.

---

2. Component hierarchy

Page is **one Server Component**. Each card section has three modes:

1. **View-only** — when the user has the read permission but not the write permission for that section.
2. **Editable** — read-only display + an inline `<form>` submitting to a server action.
3. **Locked** — after the encounter is `finished` / `cancelled`, or after a note is `signed`, the editable cards downgrade to view-only.

```
EncounterPage (Server Component)
├── AppBreadcrumbs
├── PageHeader (patient summary)
├── FlashBanner (ok / error from ?ok= / ?error=)
├── Card: Visit summary
│   └── View | <form action={updateEncounter}>
├── Card: Allergies                                    🆕
│   ├── Allergy list (view)
│   └── <form action={addAllergy}>                     (conditional)
│       + per-row <form action={removeAllergy}>
├── Card: Medications                                  🆕
│   ├── Medication list (view)
│   └── <form action={addMedication}>                  (conditional)
│       + per-row <form action={setMedicationStatus}>
├── Card: Diagnoses
│   └── [...existing]
├── Card: Vitals
│   └── [...existing]
├── Card: Clinical note
│   ├── Signed: read-only SOAP + <form action={amendNote}>
│   └── Draft: <form action={saveNoteDraft}> + <form action={signNote}>
├── Card: Documents                                    🆕
│   └── Attachment list (read-only, metadata only)
├── Card: Version history
│   └── [...existing]
└── Card: Note audit trail                             🆕
    └── Event timeline (fetched via service-role client)
```

No client components in the workspace. Every card is a pure server render. That keeps the bundle small and means RLS is always the final gate.

---

3. Note status model

```
                ┌─ save draft ─┐
draft  ──────▶│              │──▶ signed ── amend ──▶ (new row: draft)
                └──────────────┘
                       │                       ▲
                       └── sign ───────────────┘

         (previous note becomes: amended)
```

States (DB enum `note_status`):

State	Semantics	Transitions	UI
`draft`	Content in flight; any field editable by author	→ `signed` (sign), → (nothing, soft-delete not permitted)	Full editor + Sign form if `clinical:sign`
`pending_review`	**Not used in V1.** Placeholder for co-sign (attending countersigning resident)	→ `signed`, → `draft`	Hidden
`signed`	Content frozen; signer + timestamp + SHA-256 hash written to `public.signatures`	→ (nothing for this row; amendment creates a new row, this row flips to `amended`)	Read-only SOAP + Amend form if `clinical:amend`
`amended`	Superseded by a newer version (`amended_from` pointer on the new row)	(terminal)	Appears only in Version history

Invariants

- A sign must produce a matching `public.signatures` row in the same logical operation; see §5 for the transactional gap in V1.
- An amendment requires a reason ≥5 chars; reason is emitted as a semantic audit event `note.amended`.
- Version numbers are strictly increasing per encounter and pre-allocated by the app on amend (`current.version + 1`).

---

4. Versioning model

Single table (`public.encounter_notes`) with a linked-list via `amended_from`. The "current" note per encounter is the row with `status != 'amended'` and the highest `version`. The `public.clinical_notes_current` SQL view (migration `20260420000001`) exposes this as a first-class handle.

```
v1 (signed)         amended      ┐
├── signed_at       ├── signed_at │
├── signed_by       ├── signed_by │
└── amended_from    └── amended   │
     = NULL              FROM     │  chain
                         = NULL  <┘
                                  │
v2 (signed)         amended   ───┐│
├── signed_at       ├── signed_at││
├── signed_by       ├── signed_by││
└── amended_from    └── amended ◀┘│
     = v1                 FROM    │
                          = v1    │
                                 <┘
v3 (draft/signed)   CURRENT
├── ...             (status != amended)
└── amended_from
     = v2
```

Content is **copied forward** on amend (app layer) so each row is a self-contained snapshot. This is intentional:

- Auditing is trivial — the row itself carries all fields.
- Display is trivial — no diff reconstruction needed.
- Storage overhead is acceptable for clinical note sizes.

See docs/clinical-domain.md §2.6 for the design trade-off vs. the split-table alternative.

---

5. Autosave vs. manual save — recommendation

V1: manual save. `Save draft` button on the SOAP editor; no autosave, no dirty-state indicator beyond the button.

Why manual for V1:

- **Round-trip cost** — each save hits Postgres + audit trigger. Autosave-on-idle would 3-5× write rate.
- **Concurrency risk** — two authors on the same draft with autosave last-write-wins each other invisibly. Manual save + an optimistic-lock column makes conflicts explicit.
- **Signing intent** — clinicians are deliberate about when a note "settles." A visible Save → Sign flow matches their mental model.
- **Connection loss** — manual save + a browser "You have unsaved changes" beforeunload handler (v2) gives the user agency.

V2 plan when it makes sense:

- Add `updated_at` optimistic-lock check on the `saveNoteDraft` action (note.ts:53 already has `ifUnmodifiedSince`).
- Introduce autosave on a 30-second idle timer, only for the active author and only when no other row is newer. On conflict, show a merge banner.
- Keep the Save button — autosave is a safety net, not the primary flow.

Not V2: real-time collaborative editing. That's a completely different beast (OT/CRDT) and clinical notes don't typically benefit.

---

6. Audit hooks

6.1 Row-level (automatic)

`audit.log_change()` trigger is attached to:

- `public.encounter_notes` (INSERT/UPDATE/DELETE)
- `public.encounters`
- `public.allergies`
- `public.medications`
- `public.vitals`
- `public.signatures`
- `public.attachments`
- `public.diagnosis_assignments`

Every write captures `before` / `after` jsonb, `actor_id = auth.uid()`, `impersonator_id` via `current_impersonation()`, into `audit.audit_events`. No app code required.

6.2 Semantic (app-level)

Emitted via `logEventBestEffort` from server actions:

Event type	Emitted from	Details
`note.amended`	`amendNote` action (already wired)	`amended_from`, `from_version`, `to_version`, `reason`
`note.signed`	`signNote` action (planned, not yet wired)	`signature_id`, `hash_prefix`
`encounter.opened`	`openEncounter` action (planned)	`appointment_id`
`encounter.finished`	`finishEncounter` action (planned)	`open_orders`, `note_version`
`ai.draft_generated`	AI endpoint (planned)	`completion_id`, `model`, `latency_ms`

The row-level trigger is the authoritative audit. Semantic events enrich the timeline with intent (why, not just what).

6.3 In-page audit trail (§8 card)

Read from `audit.audit_events` via the **service-role** admin client (audit events have no authenticated-user SELECT policy yet). Filter: `tenant_id = current tenant`, `table_name = 'encounter_notes'`, `row_id ∈ this encounter's note ids`. Sort by `occurred_at DESC`, limit 20.

Gated on `audit:read` (today: `practice_owner`, `office_admin`). Physicians don't see the card inline; they can use `/admin/security` (future) for a full viewer.

---

7. API interactions

All mutations are Server Actions. Ordered by surface area:

Action	Permission	Side effects
`updateEncounter`	`clinical:write`	update row; if `status=finished`, set `end_at`
`addAllergy`, `removeAllergy`	`patient:write`	insert / soft-delete `public.allergies`
`addMedication`, `setMedicationStatus`	`clinical:write`	insert / update `public.medications`
`assignDiagnosis`, `removeDiagnosis`	`clinical:write`	insert / soft-delete `public.diagnosis_assignments`
`recordVitals`	`patient:write`	insert `public.vitals`
`saveNoteDraft`	`clinical:write`	upsert `public.encounter_notes`
`signNote`	`clinical:sign`	insert `public.signatures` + flip note status
`amendNote`	`clinical:amend`	insert new note (draft) + flip old to amended + `logEvent(note.amended)`

Atomicity gaps in V1 (noted in slice commits):

- `signNote` writes the signature row *then* updates the note status — two statements, not a transaction. Failure between them leaves a dangling signature.
- `amendNote` inserts the new draft *then* flips the old row — same pattern.

Both are V2 targets: wrap each pair in a `SECURITY DEFINER` RPC (`public.sign_note`, `public.amend_note`) so the DB enforces atomicity and the trigger context is consistent.

---

8. Edge cases

Case	Handling
Encounter in another tenant	RLS returns nothing → `notFound()` → 404
Allergy added by scheduler	Blocked by `patient:write` guard + RLS
Medication without dose/route	Allowed at DB (both nullable); flagged in UI with warning badge (v2)
Note signed, encounter still `in_progress`	Allowed — common when the physician signs late in the day; the Finish encounter action checks note is signed
Finish encounter with unsigned note	Blocked at app layer; "sign the note first" error
User without `clinical:amend` viewing a signed note	Amend form hidden; the read-only SOAP renders as usual
Impersonator signing a note	Blocked: `IMPERSONATION_BLOCKED_V2` strips `encounters:sign` / `notes:sign` ; the action refuses and the button is hidden
Two authors concurrently editing a draft	Last-write-wins in V1; `ifUnmodifiedSince` check planned in V2
Amendment chain length	No hard cap; regulatory interest is in the chain existing, not its length
Document upload attempted	Not in V1 — the section renders a read-only list; uploader ships with storage bucket wiring
Audit card for physician (no `audit:read`)	Card simply not rendered
Deleted patient	Encounter becomes read-only; RLS still returns encounter rows for historic access
Session expired mid-edit	Action redirects to `/login?next=/encounters/{id}` ; unsaved typed content is lost (V2 browser draft persistence)

---

9. Acceptance criteria

9.1 Header

- Patient name + MRN + DOB + age + sex + pronouns + encounter status all visible.
- Open chart button navigates to `/patients/[id]` .

9.2 Visit summary

- Editable (chief complaint, status, reason) when `clinical:write` and encounter is active.
- Setting status= `finished` sets `end_at=now()` in the same update.

9.3 Allergies

- Lists all non-deleted allergies for the patient (note: patient-scoped, not encounter-scoped).
- Add form: type (medication/food/environmental/other), substance, reaction, severity, notes.
- Remove is soft-delete. Row stays queryable for audit.
- Hidden for roles without `patient:write`.

9.4 Medications

- Lists non-deleted medications for the patient with status badge.
- Add form: display_name, dose, route, frequency, start_date, end_date.
- Per-row status transitions: active ↔ on_hold, active → stopped/completed.
- Default status on add: `active`.
- Hidden for roles without `clinical:write`.

9.5 Diagnoses

- Already shipped (insurance/diagnosis PR). Accepts ICD-10 regex, auto-ranks, cap 12.

9.6 Vitals

- Already shipped (Slice 3). Timeline + entry form.

9.7 Clinical note

- Already shipped (Slices 3 + 4). Draft editor + Sign + Amend.

9.8 Documents

- Lists attachments where `encounter_id = this encounter` (or patient-scoped if ticked).
- Displays: kind, label, mime, size, uploaded_by, signed state, effective_date.
- No download / upload yet; notes "upload ships with storage bucket wiring."

9.9 Version history

- Already shipped (Slice 4).

9.10 Note audit trail

- Visible only to `audit:read`.
- Shows up to 20 most-recent events touching this encounter's note rows.
- Each event: timestamp, actor, action (INSERT/UPDATE + semantic event_type if set), short summary.

---

10. Rollout plan

This PR:

- Allergies card (list + add + remove).
- Medications card (list + add + status transitions).
- Documents card (read-only list).
- Note audit trail card (gated read).
- This design doc.

Near-term follow-ups:

- `sign_note` / `amend_note` SECURITY DEFINER RPCs for atomicity.
- Autosave + optimistic locking on note drafts.
- Document uploader UI (needs Storage bucket + RLS).
- AI draft generation endpoint + in-workspace generation form.

Deferred:

- Patient self-view of their own audit trail (`/my/security`).
- Co-sign (`pending_review` workflow for residents).
- Collaborative real-time editing.